

1. Overview

This Artist Marketplace API is a RESTful web service designed to manage artist verification and registration. It allows public users to search for artists and submit registration requests, while administrators can approve, update, and delete artist records.

The system demonstrates multi-client support, where both a web application and a Python CLI interact with the same API.

2. Base URL

API: <http://localhost/school-app/api.php>

Client I: http://localhost/school-app/admin_tool.py

Client II: <http://localhost/school-app/index.html>

3. Authentication

Administrative actions require an API key sent via request headers: **X-Admin-Token: 1**. Required for viewing all artists, approving, editing, and deleting. If missing/invalid → **401 Unauthorized**.

4. Endpoints

4.1. Search/Verify Artist

HTTP Method: GET
Endpoint: </api.php?action=verify&username={username}>
Header: None required
Parameters: username (required)

Request Example:

```
GET /api.php?action=verify&username=Jesslyn
```

Response (Success):

```
{
  "status": "verified",
  "user": {
    "username": "Jesslyn",
    "bio": "Digital Artist"
  }
}
```

Response (Error):

```
{
  "error": "Not found"
}
```

4.2. Submit Registration Request

HTTP Method: POST
Endpoint: </api.php?action=request>
Header: Content-Type: application/json
Parameters: username, bio (required)

Request Body Example:

```
{
  "username": "Juju",
  "bio": "Sketch Artist"
}
```

Response (Success):

```
{
  "message": "Request sent!"
}
```

Response (Error):

```
{
  "error": "Username taken or pending."
}
```

4.3. Get Approved Artists (Admin)

HTTP Method: GET
Endpoint: </api.php>
Header: X-Admin-Token: 1
Parameters: None

Response (Success):

```
[
  {
    "username": "Jesslyn",
    "bio": "Digital Artist"
  }
]
```

4.4. Get Pending Requests (Admin)

HTTP Method: GET
Endpoint: </api.php?action=pending>
Header: X-Admin-Token: 1
Parameters: None

Response (Success):

```
[
  {
    "username": "new_artist",
    "bio": "Aspiring Artist"
  }
]
```

4.5. Approve Artist (Admin)

HTTP Method: POST

Endpoint: /api.php?action=approve&username={username}

Header: X-Admin-Token: 1

Parameters: username (required)

Response (Success):

```
{
  "message": "Artist approved & JSON updated!"
}
```

4.6. Update Artist Bio (Admin)

HTTP Method: PUT

Endpoint: /api.php?username={username}

Header: Content-Type: application/json,
X-Admin-Token: 1

Parameters: username, bio (required)

Request Body Example:

```
{
  "bio": "Updated Bio"
}
```

Response (Success):

```
{
  "message": "Bio updated in SQL & JSON"
}
```

4.7. Delete Approved Artist (Admin)

HTTP Method: DELETE

Endpoint: /api.php?username={username}

Header: X-Admin-Token: 1

Parameters: username (required)

Response (Success):

```
{
  "message": "Removed successfully"
}
```

4.8. Reject Pending Artist Request (Admin)

HTTP Method: DELETE

Endpoint: /api.php?action=reject&username={username}

Header: X-Admin-Token: 1

Parameters: username (required)

Response (Success):

```
{
  "message": "Removed successfully"
}
```

5. Request & Response Format

- Content-Type: application/json
- Supports: GET, POST, PUT, DELETE
- All responses are JSON

6. Data Storage

- Database: art_db
- Tables: artist (*approved users*), pending_request (*registration queue*)
- JSON File: verified_artists.json (*Automatically updated when artist is approved, edited, and/or deleted*)

7. System Flow

1. User submits HTTP request or search
2. API receives HTTP request
3. API validates input and authentication
4. API queries MySQL database
5. API updates JSON file (if needed)
6. API returns JSON response

8. Error Handling

Code	Description
400	Missing data
401	Unauthorized
409	Username taken or pending
500	Database Connection Failed